

Constrained Random Verification CRV Introduction

SyoSil, 2026

Contents

- Small Recap
- CRV Explained
- Transactions
- Example

Recap

Novel idea: Constrained Random Verification (CRV)

- AKA Coverage Driven Verification - CDV
- When the computer have to write the tests then how to produce:
 - **Produce Stimuli?**
 - **Check Correctness?**
 - **Check Completeness?**
 - If **stimuli** is random then **Correctness** and **Completeness** cannot be computed per test
- Thus, 1D approach has to be split!
 - Into individual dimensions (3D) as the computer will write test by generating **stimuli** not knowing the **completeness** and **correctness** criterias.

To produce a machine written test, then the computer needs to be able to independently:

- Produce **stimuli**
- Check/report **correctness**
- Check/report **completeness**

CRV Explained

Produce stimuli

- The stimuli is produced by expressing random **stimuli**
 - Easy to generate by declaring a test template which can be randomized
 - Hit scenarios the engineer did not anticipate
- Random data must be constrained
 - Random data is not likely to hit interesting cases -> must be constrained
 - Valid constraints, legal constraints, erroneous constraints

Check/report correctness/completeness

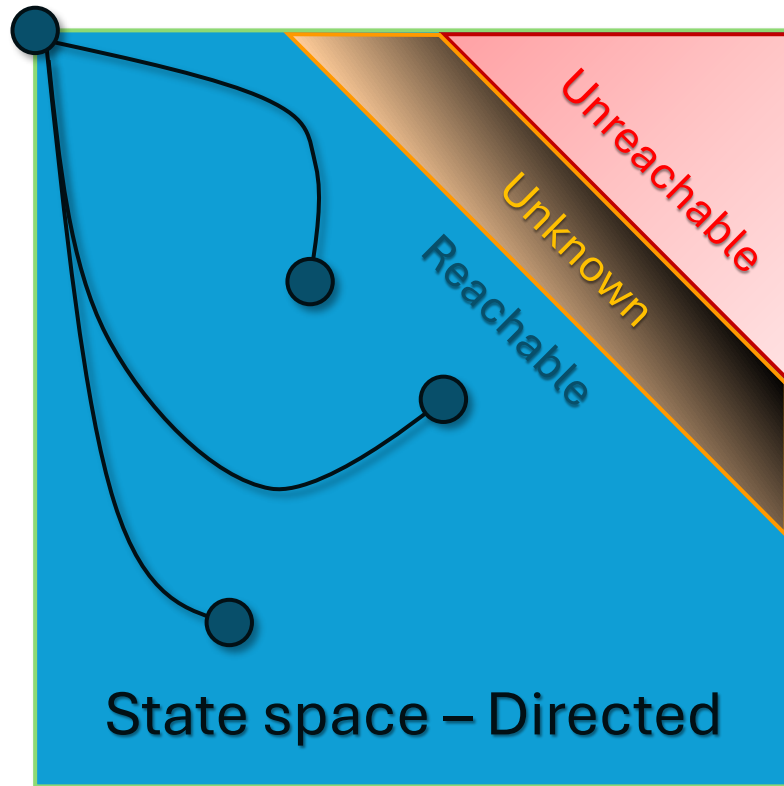
- The verification environment must support test independent
 - Functional **correctness** check e.g. by including
 - Checkers
 - Reference models
 - Functional **completeness**
 - Can be expressed as functional coverage
- Both **MUST** be independent of the test for **scalability** as the computer can potentially produce thousands of tests automatically

Compare Directed Testing vs CRV

	Directed Testing	CRV
Stimuli	• Tests are implemented manually, producing low coverage of the state space • Poor re-usability of testcases and test harness	• Test templates are implemented, producing high coverage of the state space • OOP for reuse
Correctness	• Checks build into tests individually • Maybe manual inspection	• Independent checkers • Reference models
Completeness	• Functional state space is big • Need to implement many tests to cover state space • Only the scenarios imagined by the verification engineer are verified	• Functional state space is still big • Computer writes the tests • Randomness will hit unanticipated scenarios • Write functional coverage to capture the functional state space

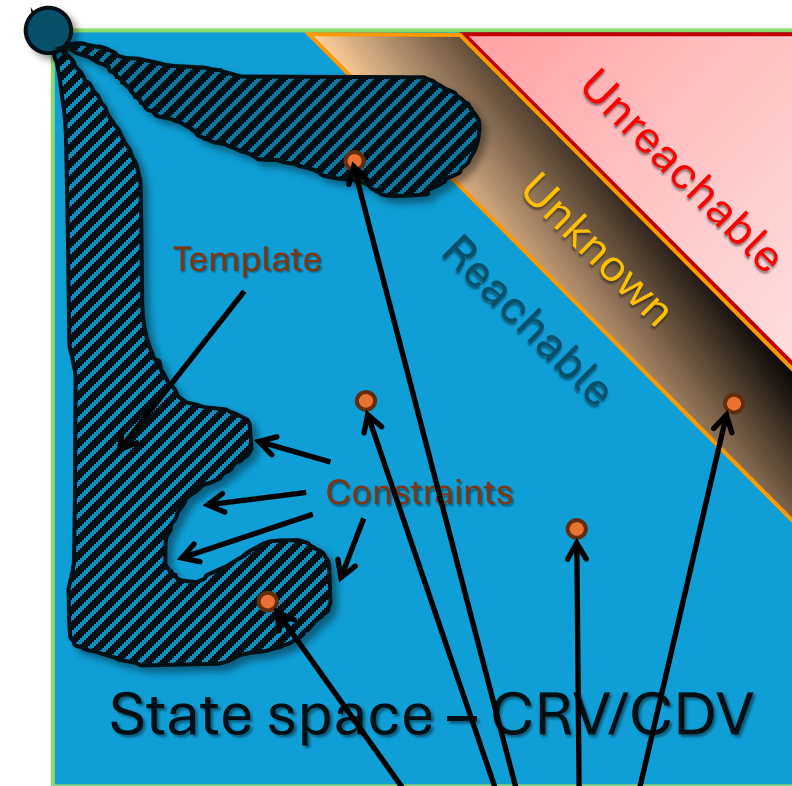
Compare Directed Testing vs CRV

Initial state



- How to reach all the states?
- How to reach unknown parts of the state space?

Initial state



- Testcase templating
- Random stimuli + constraints
- Functional coverage

How to let the computer write tests?

- How to produce random stimuli using constraints?
- Random wiggling input pins on the DUT?
 - Error prone, tedious (and boring)
 - Would break relevant protocols
- Fundamentally, device stimuli is pin wiggling
 - It is repetitive → It can be abstracted!
- How to abstract pin wiggling?
 - Answer: **Transactions!**

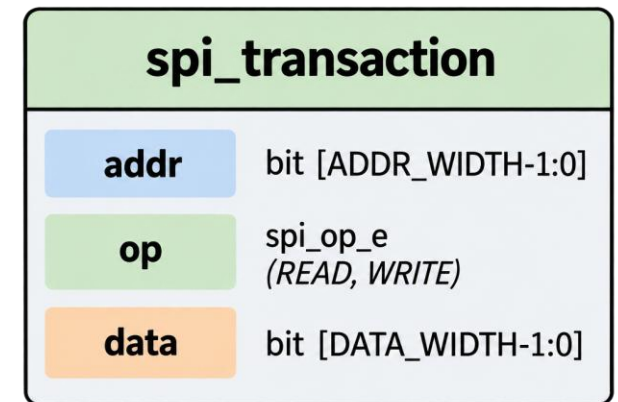
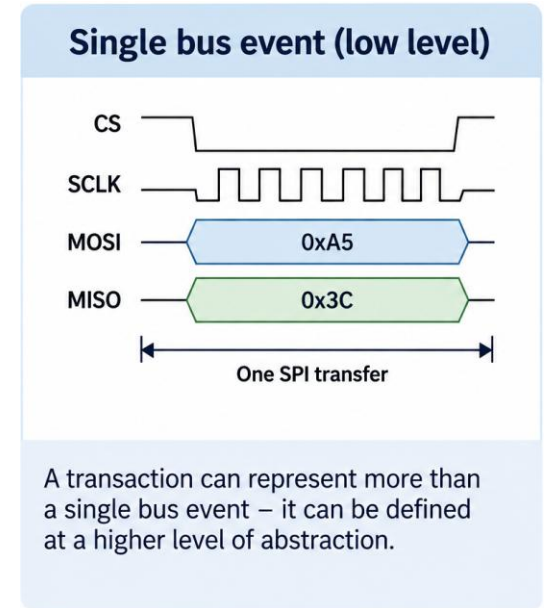
- **Key concept**

- Expressing random stimuli in test templates in CRV is randomization of transactions instead of trying to randomize pin wiggling directly
- Randomize abstract transaction → convert to pin wiggling (converter can be reused for all transactions)

Transactions

Transactions

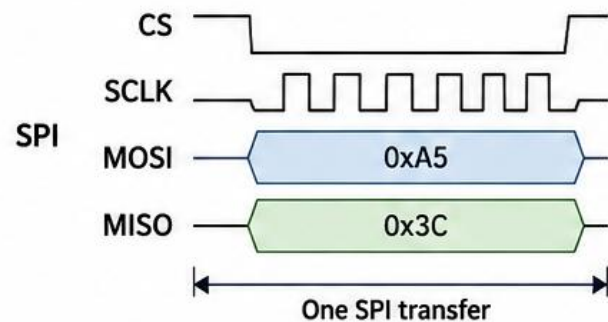
- A transaction can be a minimal description of a bus event
- A test template randomizes stimuli by randomizing transactions
- Constraints are written on transaction level, **NOT** pin level. Leverage the abstraction in the tests as well
- The computer knows how to instantiate a test template and randomize it. Thus it will produce random stimuli which the verification environment applies to the device
- A regression can be setup which executes the same test template many times with different randomizations of the transactions
 - Resulting in functional state space coverage
- Side note:
 - Functional coverage can be measured at transaction level or pin level



Transactions can be more abstract!

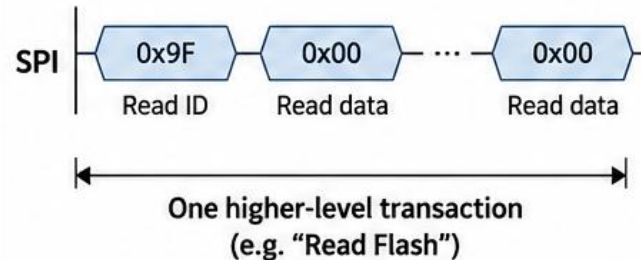
- A transaction can represent more than a single bus event — it can be defined at a higher level of abstraction.
- It may represent a sequence of related events on the same interface, e.g. several SPI transfers forming one logical operation.
- It may also represent coordinated activity across multiple interfaces — possibly of different types and even occurring in parallel.

1 Single bus event (low level)



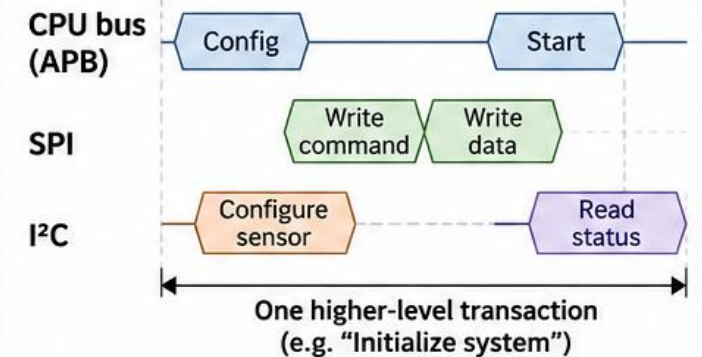
A transaction can describe a single bus event, e.g. one SPI transfer.

2 Sequence on same interface



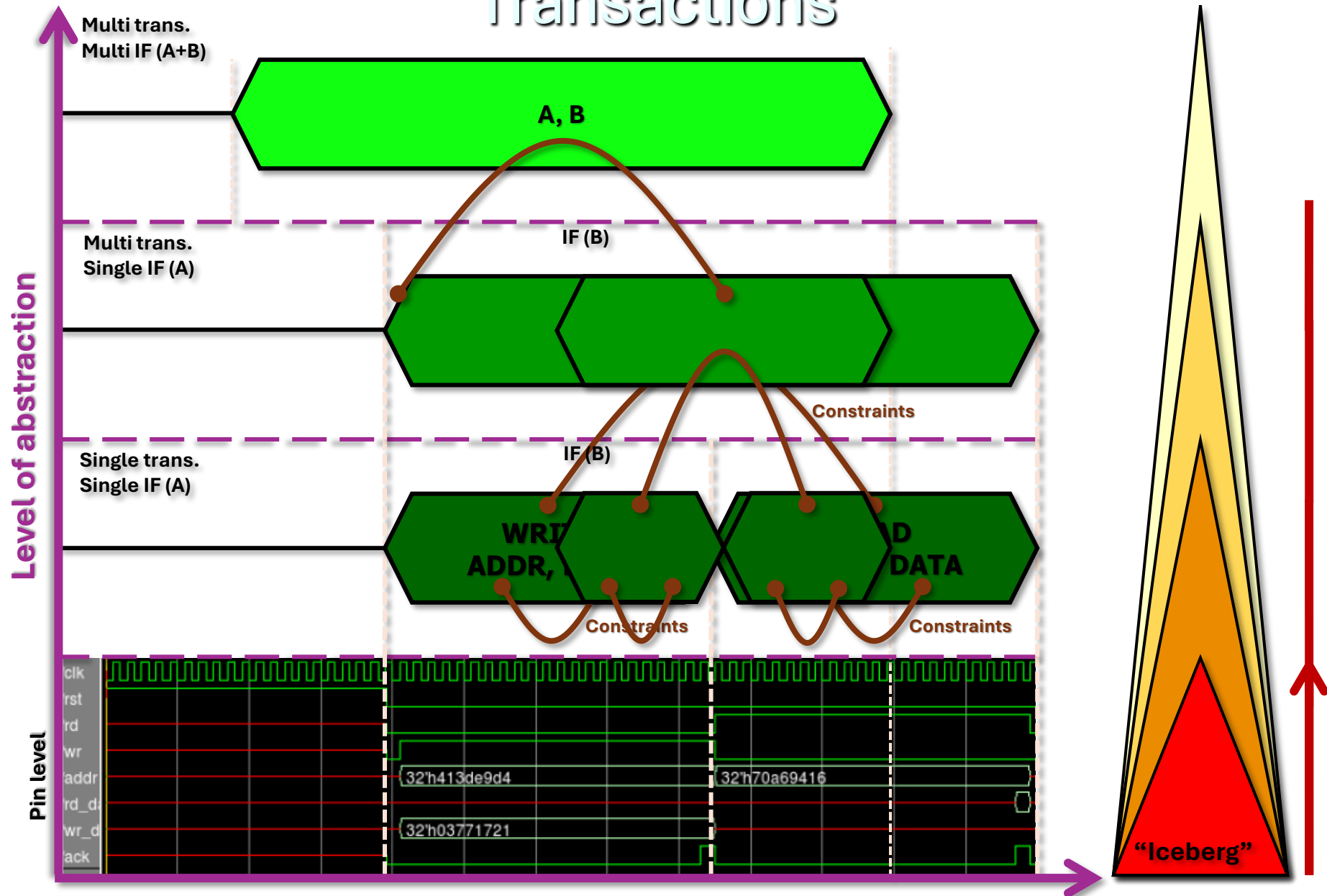
A transaction can represent a sequence of related events on the same interface, e.g. several SPI transfers forming one logical operation.

3 Across multiple interfaces



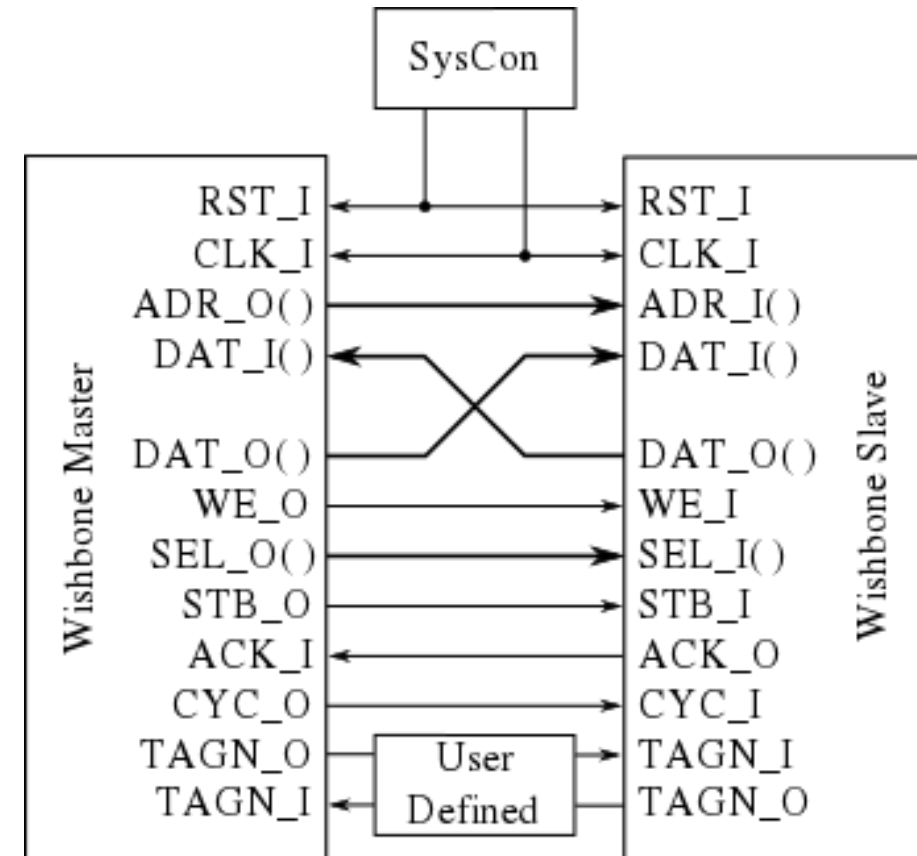
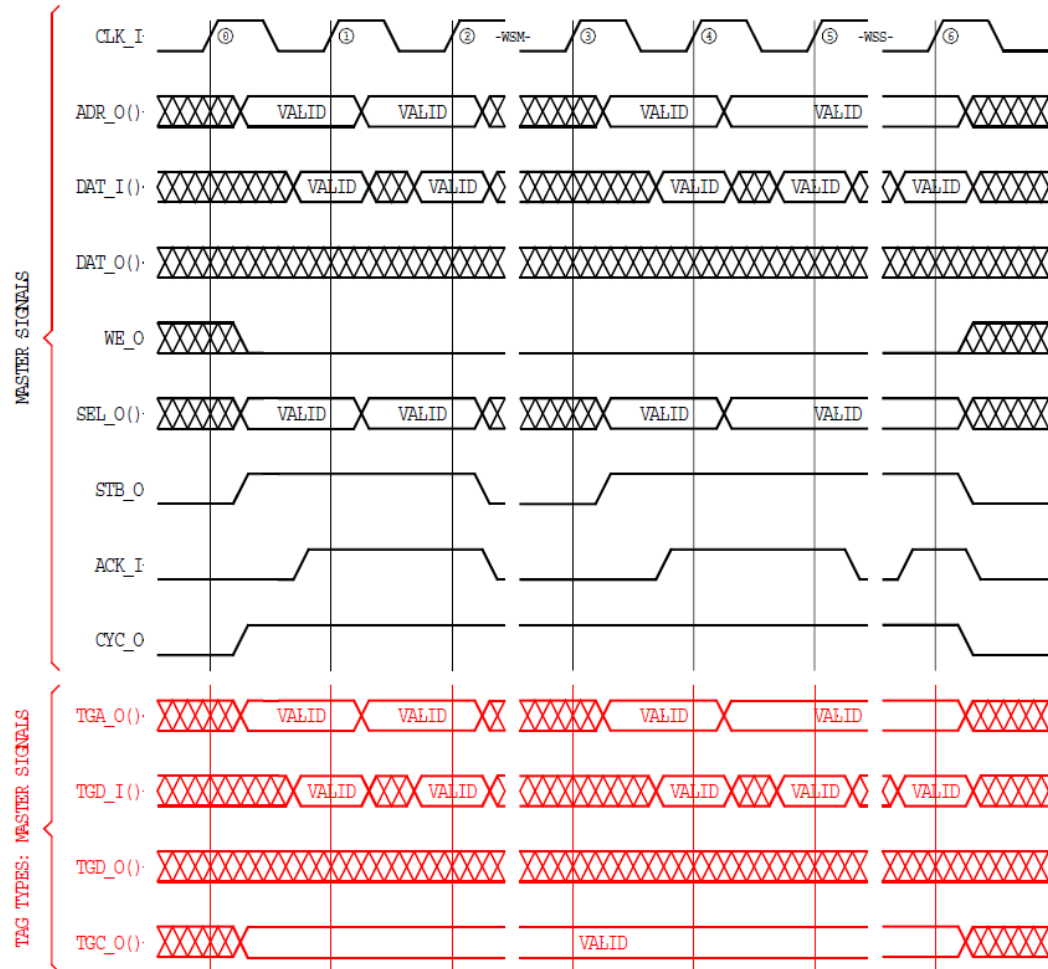
A transaction can represent coordinated activity across multiple interfaces – possibly of different types and even occurring in parallel.

Transactions



Example

CRV example with PyUVM (Wishbone)



CRV example with PyUVM (Wishbone)

```
@vsc.randobj
class cl_sdt_seq_item(uvm_sequence_item):
    """Common base sequence item for the SDT agent"""
    def __init__(self, name = "sdt_item"):
        uvm_sequence_item.__init__(self, name)

        # Transaction members
        self.data = vsc.rand_uint32_t()
        self.adr = vsc.rand_uint32_t()
        self.sel = vsc.rand_bit_t(4)
        self.op = vsc.rand_bit_t(1)
        self.slave_resp = vsc.rand_bit_t(2)

# Constraint example
    @vsc.constraint
    def c_data(self):
        self.data <= 1014
        with vsc.if_then(self.adr == 16):
            self.op == 0

    def do_copy(self, rhs):
        # Defines how copy of the SDT sequence item is done
        # Used when calling clone() of SDT sequence item

    def __eq__(self, other) -> bool:
        # Defines how sequence items are compared

    def __str__(self) -> str:
        # Defines output string when printing sequence item
```